

org.apache.spark.sql

Class GroupedData

Object

org.apache.spark.sql.GroupedData

public class **GroupedData**
extends Object

:: Experimental :: A set of methods for aggregations on a **DataFrame**, created by **DataFrame.groupBy**.

Since:

1.3.0

Method Summary

Methods

Modifier and Type	Method and Description
DataFrame	agg (Column expr, Column ... exprs) Compute aggregates by specifying a series of aggregate columns.
DataFrame	agg (Column expr, scala.collection.Seq< Column > exprs) Compute aggregates by specifying a series of aggregate columns.
DataFrame	agg (scala.collection.immutable.Map<String,String> exprs) (Scala-specific) Compute aggregates by specifying a map from column name to aggregate methods.
DataFrame	agg (java.util.Map<String,String> exprs) (Java-specific) Compute aggregates by specifying a map from column name to aggregate methods.
DataFrame	agg (scala.Tuple2<String,String> aggExpr, scala.collection.Seq<scala.Tuple2<String,String>> aggExprs) (Scala-specific) Compute aggregates by specifying a map from column name to aggregate methods.
static GroupedData	apply (DataFrame df, scala.collection.Seq<org.apache.spark.sql.catalyst.expressions.Expression> groupingExprs, org.apache.spark.sql.GroupedData.GroupType groupType)
DataFrame	avg (scala.collection.Seq<String> colNames) Compute the mean value for each numeric columns for each group.
DataFrame	avg (String... colNames) Compute the mean value for each numeric columns for each group.
DataFrame	count () Count the number of rows for each group.
DataFrame	max (scala.collection.Seq<String> colNames) Compute the max value for each numeric columns for each group.
DataFrame	max (String... colNames) Compute the max value for each numeric columns for each group.
DataFrame	mean (scala.collection.Seq<String> colNames) Compute the average value for each numeric columns for each group.
DataFrame	mean (String... colNames) Compute the average value for each numeric columns for each group.
DataFrame	min (scala.collection.Seq<String> colNames) Compute the min value for each numeric column for each group.
DataFrame	min (String... colNames) Compute the min value for each numeric column for each group.
DataFrame	sum (scala.collection.Seq<String> colNames) Compute the sum for each numeric columns for each group.
DataFrame	sum (String... colNames) Compute the sum for each numeric columns for each group.

Methods inherited from class Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

apply

```
public static GroupedData apply(DataFrame df,
                                scala.collection.Seq<org.apache.spark.sql.catalyst.expressions.Expression> groupingExprs,
                                org.apache.spark.sql.GroupedData.GroupType groupType)
```

agg

```
public DataFrame agg(Column expr,
                   Column... exprs)
```

Compute aggregates by specifying a series of aggregate columns. Note that this function by default retains the grouping columns in its output. To not retain grouping columns, set `spark.sql.retainGroupColumns` to false.

The available aggregate methods are defined in [functions](#).

```
// Selects the age of the oldest employee and the aggregate expense for each department
```

```
// Scala:
import org.apache.spark.sql.functions._
df.groupBy("department").agg(max("age"), sum("expense"))
```

```
// Java:
import static org.apache.spark.sql.functions.*;
df.groupBy("department").agg(max("age"), sum("expense"));
```

Note that before Spark 1.4, the default behavior is to NOT retain grouping columns. To change to that behavior, set config variable `spark.sql.retainGroupColumns` to false.

```
// Scala, 1.3.x:
df.groupBy("department").agg($"department", max("age"), sum("expense"))
```

```
// Java, 1.3.x:
df.groupBy("department").agg(col("department"), max("age"), sum("expense"));
```

Parameters:

expr - (undocumented)

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

mean

```
public DataFrame mean(String... colNames)
```

Compute the average value for each numeric columns for each group. This is an alias for `avg`. The resulting [DataFrame](#) will also contain the grouping columns. When specified columns are given, only compute the average values for them.

Parameters:

colNames - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

max

```
public DataFrame max(String... colNames)
```

Compute the max value for each numeric columns for each group. The resulting [DataFrame](#) will also contain the grouping columns. When specified columns are given, only compute the max values for them.

Parameters:

colNames - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

avg

```
public DataFrame avg(String... colNames)
```

Compute the mean value for each numeric columns for each group. The resulting [DataFrame](#) will also contain the grouping columns. When specified columns are given, only compute the mean values for them.

Parameters:

colNames - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

min

```
public DataFrame min(String... colNames)
```

Compute the min value for each numeric column for each group. The resulting [DataFrame](#) will also contain the grouping columns. When specified columns are given, only compute the min values for them.

Parameters:

colNames - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

sum

```
public DataFrame sum(String... colNames)
```

Compute the sum for each numeric columns for each group. The resulting [DataFrame](#) will also contain the grouping columns. When specified columns are given, only compute the sum for them.

Parameters:

colNames - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

agg

```
public DataFrame agg(scala.Tuple2<String,String> aggExpr,  
    scala.collection.Seq<scala.Tuple2<String,String>> aggExprs)
```

(Scala-specific) Compute aggregates by specifying a map from column name to aggregate methods. The resulting **DataFrame** will also contain the grouping columns.

The available aggregate methods are avg, max, min, sum, count.

```
// Selects the age of the oldest employee and the aggregate expense for each department  
df.groupBy("department").agg(  
    "age" -> "max",  
    "expense" -> "sum"  
)
```

Parameters:

aggExpr - (undocumented)

aggExprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

agg

```
public DataFrame agg(scala.collection.immutable.Map<String,String> exprs)
```

(Scala-specific) Compute aggregates by specifying a map from column name to aggregate methods. The resulting **DataFrame** will also contain the grouping columns.

The available aggregate methods are avg, max, min, sum, count.

```
// Selects the age of the oldest employee and the aggregate expense for each department  
df.groupBy("department").agg(Map(  
    "age" -> "max",  
    "expense" -> "sum"  
))
```

Parameters:

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

agg

```
public DataFrame agg(java.util.Map<String,String> exprs)
```

(Java-specific) Compute aggregates by specifying a map from column name to aggregate methods. The resulting **DataFrame** will also contain the grouping columns.

The available aggregate methods are avg, max, min, sum, count.

```
// Selects the age of the oldest employee and the aggregate expense for each department
import com.google.common.collect.ImmutableMap;
df.groupBy("department").agg(ImmutableMap.of("age", "max", "expense", "sum"));
```

Parameters:

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

agg

```
public DataFrame agg(Column expr,
                     scala.collection.Seq<Column> exprs)
```

Compute aggregates by specifying a series of aggregate columns. Note that this function by default retains the grouping columns in its output. To not retain grouping columns, set `spark.sql.retainGroupColumns` to false.

The available aggregate methods are defined in [functions](#).

```
// Selects the age of the oldest employee and the aggregate expense for each department
```

```
// Scala:
```

```
import org.apache.spark.sql.functions._
df.groupBy("department").agg(max("age"), sum("expense"))
```

```
// Java:
```

```
import static org.apache.spark.sql.functions.*;
df.groupBy("department").agg(max("age"), sum("expense"));
```

Note that before Spark 1.4, the default behavior is to NOT retain grouping columns. To change to that behavior, set config variable `spark.sql.retainGroupColumns` to false.

```
// Scala, 1.3.x:
```

```
df.groupBy("department").agg($"department", max("age"), sum("expense"))
```

```
// Java, 1.3.x:
```

```
df.groupBy("department").agg(col("department"), max("age"), sum("expense"));
```

Parameters:

expr - (undocumented)

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

count

```
public DataFrame count()
```

Count the number of rows for each group. The resulting `DataFrame` will also contain the grouping columns.

Returns:

(undocumented)

Since:

1.3.0

mean

```
public DataFrame mean(scala.collection.Seq<String> colNames)
```

Compute the average value for each numeric columns for each group. This is an alias for `avg`. The resulting `DataFrame` will also contain the grouping columns. When specified columns are given, only compute the average values for them.

Parameters:

`colNames` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

max

```
public DataFrame max(scala.collection.Seq<String> colNames)
```

Compute the max value for each numeric columns for each group. The resulting `DataFrame` will also contain the grouping columns. When specified columns are given, only compute the max values for them.

Parameters:

`colNames` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

avg

```
public DataFrame avg(scala.collection.Seq<String> colNames)
```

Compute the mean value for each numeric columns for each group. The resulting `DataFrame` will also contain the grouping columns. When specified columns are given, only compute the mean values for them.

Parameters:

`colNames` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

min

```
public DataFrame min(scala.collection.Seq<String> colNames)
```

Compute the min value for each numeric column for each group. The resulting `DataFrame` will also contain the grouping columns. When specified columns are given, only compute the min values for them.

Parameters:

`colNames` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

sum

```
public DataFrame sum(scala.collection.Seq<String> colNames)
```

Compute the sum for each numeric columns for each group. The resulting [DataFrame](#) will also contain the grouping columns. When specified columns are given, only compute the sum for them.

Parameters:

colNames - (undocumented)

Returns:

(undocumented)

Since:

1.3.0